



Practica 4

[Practica4_2026.pdf](#)

[Copia de 04 Explicación de práctica.pdf](#)

Ejercicio 1: a) Tome una de las variables de la línea 3 del siguiente código e indique y defina cuales son sus atributos:

-Variable A atributos:

-Nombre: a

-Alcance: estatico, linea 4 a 16

-Tipo: Integer

-L-valor: automatico

-R-valor: indefinido

-TV: linea 1 a 16

```
1. Procedure Practica4();
2. var
3. a,i:integer
4. p:puntero
5. Begin
6. a:=0;
7. new(p);
8. p:= ^i
9. for j:=1 to 9 do
10 .a:=a+i;
11 .end;
12 .....
13 .p:= ^a;
14 .....
15. dispose(p);
16. end;
```

b) Compare los atributos de la variable del punto a) con los atributos de la variable de la línea 4. Que dato contiene esta variable?

-Variable p atributos:

- Nombre: p
- Alcance: p: 5-16 — p[^]: 5-16
- Tipo: puntero
- L-valor: p: automatico — p[^]: dinamico
- R-valor: p: nil — p[^]: indefinido
- TV: p: 1-16 — p[^]: 7-15

Ejercicio 2:

- a. Indique cuales son las diferentes formas de inicializar una variable en el momento de la declaración de la misma.

-Ignorar el problema: se inicializa con el valor que hay en memoria (cadena de bits). Puede llevar a errores

-Inicializacion por defecto: las variables se inicializan con un valor por defecto, enteros en 0, caracteres en blanco, etc

-Inicializacion en declaracion: la variable se inicializa en el mismo momento de la declaracion, por ejemplo, `int 1 = 0`

- b. Analice en los lenguajes: Java, C, Python y Ruby las diferentes formas de inicialización de variables que poseen. Realice un cuadro comparativo de esta característica.

Forma de inicialización	Java	C	Python	Ruby
Ignorar el problema (valor basura)	No permitido (error si no se inicializa antes de usar)	Sí	No permitido (error si se usa sin asignar)	No
Inicialización por defecto	Sí	Sí (solo para variables globales y estaticas)	No	No
Inicialización en la declaración	<code>int x = 5;</code>	<code>int x = 5;</code>	<code>x = 5</code>	<code>x = 5</code>

Ejercicio 3: Explique los siguientes conceptos asociados al atributo l-valor de una:

- a. Variable estática.
- b. Variable automática o semiestática.
- c. Variable dinámica.
- d. Variable semidinámica.

De al menos un ejemplo de cada uno. Investigue sobre que tipos de variables respecto de su l-valor hay en los lenguajes C y Ada.

Variable	Definición	Ejemplo en C	Ejemplo en Ada
Variable estática	Su tiempo de vida abarca toda la ejecución del programa. Se asigna al inicio y se libera al final. Conserva	<code>static int x = 0;</code>	No corresponde

Variable	Definición	Ejemplo en C	Ejemplo en Ada
	su valor entre llamadas si es local.		
Variable automática o semiestática	Su tiempo de vida está limitado al bloque donde se declara. Se crea al entrar y se destruye al salir.	<code>int x = 10;</code>	<code>Auto_Var: Integer := 20;</code>
Variable dinámica	Se crea y destruye durante la ejecución, generalmente en la heap. Su tiempo de vida es controlado explícitamente por el programador.	<code>int* dynamic_var;</code>	<code>P : access Integer := new Integer'(20);</code>
Variable semidinámica	Son variables cuyo tamaño puede ser dinámico, pero su dirección de memoria es estática	No corresponde	<code>Array: Array(1..n) of Integer;</code>

Ejercicio 4:

a. ¿A qué se denomina variable local y a qué se denomina variable global?

-Variable global: Son todas las referencias a variables que se crean en el programa principal

-Variable local: Son todas las referencias a variables que se crean dentro de una entidad (modulo o submodulo)

b. ¿Una variable local puede ser estática respecto de su l-valor? En caso afirmativo dé un ejemplo

-Si es posible, por ejemplo en C puedes declarar una variable como static y va a ser estática respecto de su l-valor

c. Una variable global ¿siempre es estática? Justifique la respuesta.

-Puede no serlo, en C si no declaras a la variable global como "static" y en su lugar usas la declaración de "extern" la variable va a ser global y no estática

d. Indique qué diferencia hay entre una variable estática respecto de su l-valor y una constante

Característica	Variable estática	Constante
L-valor (dirección)	Dirección de memoria donde se almacena su valor	No tienen un l-valor tradicional, no ocupan una posición de memoria de la misma forma que las variables (se trata como valor literal)
Valor/Persistencia	Mantiene su valor pero es modificable dentro de operaciones en funciones	Mantiene su valor y no se puede cambiar durante la ejecución
Uso	Cuando se necesita conservar y actualizar un valor	Cuando se necesita un valor fijo

Ejercicio 5:

a. En Ada hay dos tipos de constantes, las numéricas y las comunes. Indique a que se debe dicha clasificación.

-La clasificacion se debe a como y cuando se define su valor (de que forma y cuando se utiliza)

-Constantes numericas: Se definen mediante expresiones numericas y representan enteros, reales, etc. No tienen un tipo fijo y se adaptan al contexto donde se utilizan. La ligadura se produce durante la compilacion, por lo que los valores de las constantes se conocen antes de la ejecucion.

-Constantes comunes: Se definen mediante caracteres, cadenas de texto y tienen una tipo explicito para cada tipo de constante. Se evalujan en tiempo de ejecucion, por lo que el valor de la constante se desconoce antes de la misma y se conoce solo despues del inicio del programa

b. En base a lo respondido en el punto a), determine el momento de ligadura de las constantes del siguiente código:

H: constant Float:= 3,5;

-Constante numerica de tipo float con valor 3,5, su momento de ligadura es durante compilacion

I: constant:= 2

-Constante numerica, sin tipo y con valor 2. La ligadura se produce durante la compilacion

K : constant float: = H*I;

-Utiliza tanto H como I, siendo H e I dos constantes numericas, K deberia ser tambien numerica y su valor sera dado en compilacion (despues de H e I). La ligadura se va a producir durante la compilacion

Ejercicio 6: Sea el siguiente archivo con funciones de C:

```
Archivo.c
{ int x = 1; (1)
  int func1(){
    int i;
    for (i=0; i < 4 ; i++) x = x+1;
  }
  int func2(){
    int i, j;
    /*sentencias que contienen declaraciones y
    sentencias que no contienen declaraciones*/
    .....
    for (i=0; i<3 ; i+=) j= func1 + 1;
  }
}
```

Analice si llegaría a tener el mismo comportamiento en cuanto a aloación de memoria, sacar la declaración (1) y colocar dentro de func1() la declaración static int x=1;

-El comportamiento en cuanto a alocacion de memoria es el mismo ya que tener "static int x = 1: dentro de la funcion haria que se comporte como una variable estatica, donde su memoria se mantiene asignada durante toda la vida del programa. En caso de tenerla como "int x = 1"

como variable global dentro del programa principal seria lo mismo, ya que su memoria se asigna una vez al iniciar el programa.

Ejercicio 7: Sea el siguiente segmento de código escrito en Java, indique para los identificadores si son globales o locales.

```
Clase Persona {
    public long id
    public string nombreApellido
    public Domicilio domicilio
    private string dni;
    public string fechaNac;
    public static int cantTotalPersonas;
    //Se tienen los getter y setter de cada una de las variables
    //Este método calcula la edad de la persona a partir de la fecha de nacimiento
    public int getEdad(){
        public int edad=0;
        public string fN =
            this.getFechaNac();
        ...
        ...
        return edad;
    }
}

Clase Domicilio {
    public long id;
    public static int nro
    public string calle
    public Localidad loc;
    //Se tienen los getter y setter de cada una de las variables
}
```

-Identificadores:

-Identificadores globales: son aquellos que son variables estáticas de clase, aquellos que se declaran a nivel de la clase y pueden ser accedidos sin necesidad de crear una instancia de clase.

-cantTotalPersonas → estático en clase Persona

-nro → estático en clase Domicilio

-Identificador local: son variables de instancia no estáticas que se declaran dentro de la clase y solo pueden ser accedidos a través de una instancia de la clase

-id, nombreApellido, domicilio, dni, fechaNac → v.i de clase Persona

-id, calle, loc → v.i de clase Domicilio

Ejercicio 8: Sea el siguiente ejercicio escrito en Pascal

```

1- Program Uno;
2- type tpuntero= ^integer;
3- var mipuntero: tpuntero;
4- var i:integer;
5- var h:integer;
6- Begin
7-     i:=3;
8-     mipuntero:=nil;
9-     new(mipuntero);
10-    mipuntero^:=i;
11-    h:= mipuntero^+i;
12-    dispose(mipuntero);
13-    write(h);
14-    i:= h- mipuntero;
15- End.

```

a. **Indique el rango de instrucciones que representa el tiempo de vida de las variables i, h y mipuntero.**

-Tiempo de vida de:

- i: línea 1-15
- h: línea 1-15
- mipuntero: línea 1-15
- mipuntero^: línea 9-15

b. **Indique el rango de instrucciones que representa el alcance de las variables i, h y mipuntero.**

-Alcance de:

- i: línea 5-15
- h: línea 6-15
- mipuntero: 4-15
- mipuntero^: 4-15

c. **Indique si el programa anterior presenta un error al intentar escribir el valor de h. Justifique**

-No presenta ningún error ya a "h" se le asigna la suma de lo que vale mipuntero^ (que antes se le asignó lo que vale i, o sea 3) y lo que vale i (3). Esto se hace antes del dispose por lo que imprimiría 6 correctamente

d. **Indique si el programa anterior presenta un error al intentar asignar a i la resta de h con mipuntero. Justifique**

-Si presenta un error ya que mipuntero tiene el valor nil debido a que en la línea 12 se le hizo un dispose. Estarías haciendo int - nil y eso no se puede

e. **Determine si existe otra entidad que necesite ligar los atributos de alcance y tiempo de vida para justificar las respuestas anteriores. En ese caso indique cuál es la entidad y especifique su tiempo de vida y alcance.**

-La otra entidad que existe es el programa principal en si mismo. Este va a tener un alcance global y su tiempo de vida es desde la linea 1 hasta la 15

f. **Especifique el tipo de variable de acuerdo a la ligadura con el l-valor de las variables que encontró en el ejercicio.**

-i es automatica de tipo integer

-h es automatica de tipo integer

-mipuntero es automatica de tipo puntero y mipuntero^ es dinamica de tipo integer

Ejercicio 9: Elija un lenguaje y escriba un ejemplo:

a. En el cual el tiempo de vida de un identificador sea mayor que su alcance

b. En el cual el tiempo de vida de un identificador sea menor que su alcance

c. En el cual el tiempo de vida de un identificador sea igual que su alcance

-Ejemplo de la exp practica en C:

```
1. static int aux;
2. int v2;
3. static int fun2()
4. {extern int v1;
5.  aux=aux+1;
6. }
7. int fun3()
8. { int aux;
9.  aux=aux+1;
10.}
```

Identificador	Lvalor	Rvalor	Alcance	T. vida	Ejemplo de:
aux	estática	0	1-8 10 →	1-10	B
v2	automática	0	3-10	1-10	A
fun2		3-10	4-10	3-6	B
v1	automática	indef	5-6	3-6	A
fun3		7-10	7-10 (creo que esta incorrecto en la diapo deberia ser 8-10)	7-10	C
aux	automática	indef	8-10	7-10	B

Ejercicio 10: Si tengo la siguiente declaración al comienzo de un procedimiento:

```
int c; en C
var c:integer; en Pascal
c: integer; en ADA
```

Y ese procedimiento NO contiene definiciones de procedimientos internos. ¿Puedo asegurar que el alcance y el tiempo de vida de la variable "c" es siempre todo el procedimiento en donde se encuentra definida?. Analícelo y justifique la respuesta, para todos los casos.

-Depende de si hay nuevas declaraciones con el mismo nombre (identificador "c" en este caso) ya que se te enmascarían y el alcance se vería claramente afectado (como por ejemplo aux en el punto 9 que ocurre en C o ADA). En pascal si sería el mismo ya que Pascal no permite redeclarar variables fuera de funciones

-En cuanto al tiempo de vida si se puede asegurar que corresponde a la estructura donde este definida mientras este declarada dentro de un bloque respectivo en cada lenguaje (declare en Ada, {} en C, procedimiento en Pascal)

Ejercicio 11: a) Responda Verdadero o Falso para cada opción. El tipo de dato de una variable es:

I) Un string de caracteres que se usa para referenciar a la variable y operaciones que se pueden realizar sobre ella.

-Falso. El tipo de dato no es un string de caracteres que referencia a la variable, lo que referencia a la variable por su nombre es el identificador, no el tipo.

II) Conjunto de valores que puede tomar y un rango de instrucciones en el que se conoce el nombre.

-Falso, es un conjunto de valores que puede tomar pero no es un rango de instrucciones en el que se conoce el nombre, eso sería el alcance.

III) Conjunto de valores que puede tomar y lugar de memoria asociado con la variable.

-Falso, el tipo de valores que puede tomar es correcto pero no es un lugar de memoria asociado con la variable, eso es el l-valor.

IV) Conjunto de valores que puede tomar y conjunto de operaciones que se pueden realizar sobre esos valores.

-Verdadero, eso es el tipo de dato.

b) Escriba la definición correcta de tipo de dato de una variable.

-Conjunto de valores que puede tomar una variable y conjunto de operaciones que se pueden realizar sobre esa misma.

Ejercicio 12: Sea el siguiente programa en ADA, completar el cuadro siguiente indicando para cada variable de que tipo es en cuanto al momento de ligadura de su l-valor, su r-valor al momento de aloación en memoria y para todos los identificadores cuál es su alcance y cual es su el tiempo de vida. Indicar para cada variable su r-valor al momento de aloación en memoria

Aclaracion:

Ident.= Identificador / Tipo es el tipo de la variable respecto del l-value

T.V. = Tiempo de Vida / r-valor debe ser tomado al momento de la aloación en memoria.

El alcance de los identificadores debe indicarse desde la línea siguiente a su declaración.


```

1.   with text_io; use text_io;
2.   Procedure Main is;
3.     type vector is array(integer range <>);
4.     a, n, p: integer;
5.     v1: vector(1..100);
6.     c1: constant integer:=10;
7.     Procedure Uno is;
8.       type puntero is access integer;
9.       v2: vector(0..n);
10.      c1, c2: character;
11.      p, q: puntero;
12.      begin
13.        7.5.1. n:=4;
14.        7.5.2. v2(n):= v2(1) + v1(5);
15.        7.5.3. p:= new puntero;
16.        7.5.4. q:= p;
17.        7.5.5. ....
18.        7.5.6. free p;
19.        7.5.7. ....
20.        7.5.8. free q;
21.        7.5.9. ....
22.      end;
23.    begin
24.      n:=5;
25.      ....
26.      11. Uno;
27.      12. a:= n + 2;
28.      13. ....
29.    end

```

Ident.	Tipo	r-valor	Alcance
a (línea 4)	automática	basura	5-29
n (línea 4)	automatica	basura	5-29
p (línea 4)	automatica	basura	5-29
v1 (línea 5)	automatica	basura	6-29
c1 (línea 6)	automatica	basura	6-29
v2 (línea 9)	semidinamica	basura	10-22
c1 (línea 10)	automatica	basura	11-22
c2 (línea 10)	automatica	basura	11-22
p (línea 11)	automatica	nil	12-22
p^	dinamica	basura	12-22
q (línea 11)	automatica	nil	12-22
q^	dinamica	basura	12-22
Main (línea 2)			3-29
Uno (línea 7)			8-29

Ejercicio 13: El nombre de una variable puede condicionar (Justifique la respuesta):

a) Su tiempo de vida.

-No condiciona su tiempo de vida, su T.V no depende de su nombre.

b) Su alcance.

-Si puede condicionar su alcance ya que el alcance es el rango de instrucciones en el que se conoce el nombre. Además puede haber redeclaraciones

c) Su r-valor.

-No condiciona su r-valor. El r-valor es el valor que se le asigna a la variable y se accede a él a través del nombre de la misma, pero no lo condiciona

d) Su tipo.

-No condiciona su tipo, puedes tener una variable pepe que sea un array de enteros y estaría correcto, pero no es una buena idea utilizar nombres no-descriptivos

Ejercicio 14: Sean los siguientes archivos en C, los cuales se compilan juntos

Indicar para cada variable de qué tipo es en cuanto al momento de ligadura de su l-valor.

Indicar para cada identificador cuál es su alcance y cuál es su tiempo de vida.

Indicar para cada variable su r-valor al momento de asignación en memoria

Aclaración:

Ident. = Identificador

T.V. = Tiempo de Vida

r-valor debe ser tomado al momento de la asignación en memoria

El alcance de los identificadores debe indicarse desde la línea siguiente a su declaración.

```

ARCHIVO1.C
1.  int v1;
2.  int a;
3.  int fun2 ()
4.  { int v1, y;
5.    for(y=0; y<8; y++)
6.    { extern int v2;
7.      ...
8.    }
9.  }
10. main()
11. {static int var3;
12.  extern int v2;
13.  int v1, y;
14.  for(y=0; y<10; y++)
15.  { char var1='C';
16.    a=8v1;
17.  }
18. }
ARCHIVO2.C
19. static int aux;
20. int v2;
21. static int fun2()
22. { extern int v1;
23.   aux=aux+1;
24.   ...
25. }
26. int fun3()
27. { int aux;
28.   aux=aux+1;
29.   ...
30. }

```

Ident.	Tipo	r-valor	Alcance	T.V.
v1 (línea 1)	automatica	basura	2-4 → 9-12 → 21-23	1-28
a (línea 2)	automatica	nil	3-16	1-28
a^	dinamica	basura	3-16	15-28
fun2 (línea 3)			4-16	3-16
v1' (línea 4)	automatica	basura	5-8	3-8
y (línea 4)	automatica	basura	5-8	3-8
main (línea 9)			10-16	9-16
var3 (línea 10)	estatica	0	11-16	<1-28
v1'' (línea 12)	automatica	basura	13-16	9-16
y (línea 12)	automatica	basura	13-16	9-16
var1 (línea 14)	automatica	basura	15-16	13-16
aux (línea 17)	estatica	0	18-25	<1-28
v2 (línea 18)	automatica	basura	7-8 → 12-16 → 19-28	1-28
fun2 (línea 19)			20-23	19-23
fun3 (línea 24)			25-28	24-28
aux (línea 25)	automatica	basura	26-28	24-28

Ejercicio 15: Para javascript investigue la diferencia semántica para declarar una variable utilizando los modificadores const, var, let y la ausencia de cualquiera de estos. Compárelo con un lenguaje de su preferencia.

-Modificadores en JS:

-const: declara una variable que no cambiara de valor en todo el programa

-var: es la version antigua de let. Es una variable que puede cambiar y tiene alcance de funcion o global, es decir, si esta declarada en una funcion va a ser reconocida en toda la funcion y si esta declara de forma global es conocida en todo el programa

-let: al igual que var es una variable que puede cambiar de valor pero esta tiene alcance a nivel de bloque, es decir, si esta dentro de un while, for, if, etc solo sera reconocida dentro de ese bloque y por fuera no

-sin declaracion: si se declara una variable sin ninguna palabra clave de las anteriores se vuelve una variable global si esta fuera de una funcion

-Comparacion con Java:

Concepto	JavaScript	Java
Tipado	Dinámico (no se declara tipo)	Estático (requiere tipo)
Declaración básica	let x = 10;	int x = 10;
Constante	const x = 10; (no se puede reasignar)	final int x = 10;
Alcance de bloque	let, const	Sí (todas las variables)
Alcance de función	var	No existe

Concepto	JavaScript	Java
Variable global	Posible (incluso sin declarar, mala práctica)	No permitido
Redeclaración	var permite redeclarar	No permitido
Seguridad	Más flexible, propenso a errores	Más estricto y seguro